PYTHON FOR BEGINNERS · SESSION 6

Loop Practice

Practise nested loops, print patterns, and write number programs: factorial, prime numbers and Fibonacci.

Recap

Ask the class:

- What is the difference between `break` and `continue`?
- Which numbers does `range(1, 5)` give?
- When do you choose a `while` loop instead of a `for` loop?

Nested loops: how they run

For **every round of the outer loop**, the inner loop runs **from start to finish**.

```
for i in range(1, 3):
    for j in range(1, 4):
        print("i =", i, " j =", j)
```

OUTPUT

```
i = 1  j = 1
i = 1  j = 2
i = 1  j = 3
i = 2  j = 1
i = 2  j = 2
i = 2  j = 3
```

OUTER I	INNER J RUNS	LINES PRINTED
1	1, 2, 3	3
2	1, 2, 3	3

- The outer loop handles the **rows**.
- The inner loop handles the **columns** in each row.
- Total rounds = outer rounds x inner rounds (here 2 x 3 = 6).

Ask

How many lines would print if the outer loop was `range(1, 6)` and the inner loop was `range(1, 5)`?

Pattern programs

Every pattern is built the same way: the **outer loop counts the rows**, and **inside each row** you print spaces, stars or numbers and then move to the next line.

A rectangle of stars

```
for row in range(3):
    for col in range(5):
        print("*", end="")
    print()
```

OUTPUT

```
*****
*****
*****
```

- `end=""` keeps the stars on the **same line**.
- The empty `print()` after the inner loop **starts a new line**.

Right triangle

```
for i in range(1, 6):
    print("*" * i)
```

OUTPUT

```
*
**
***
****
*****
```

Row `i` has `i` stars. String repetition (`"*" * i`) does the inner loop for you.

Inverted triangle

```
for i in range(5, 0, -1):
    print("*" * i)
```

OUTPUT

```
*****
****
***
**
*
```

Ask

Which part of the program makes the triangle shrink?

Number triangle

```
for i in range(1, 6):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

OUTPUT

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

The same number in each row

```
for i in range(1, 6):
    print(str(i) * i)
```

OUTPUT

```
1
22
333
4444
55555
```

Pyramid

Each row needs **spaces first**, then an **odd number of stars** (1, 3, 5, ...).

```
n = 4
for i in range(1, n + 1):
    print(" " * (n - i) + "*" * (2 * i - 1))
```

OUTPUT

```
   *
  ***
 *****
*******
```

Pattern recipe

STEP	QUESTION TO ASK
1	How many **rows** are there? That is the outer loop.
2	In row `i`, how many **spaces** come first?
3	In row `i`, how many **stars or numbers** are printed?
4	Do I need `end=""`, or can I build the row as one string?

Factorial

The factorial of `n` is the **product of all numbers from 1 to n**. So `5! = 5 x 4 x 3 x 2 x 1 = 120`. By definition `0! = 1`.

With a for loop

```
n = 5
fact = 1
for i in range(1, n + 1):
    fact *= i
print(n, "! =", fact)
```

OUTPUT

```
5 ! = 120
```

- Start `fact` at **1**, not 0 (anything times 0 is 0).
- Set it **before** the loop, so it is not reset in every round.

With a while loop

```
n = 5
fact = 1
while n > 1:
    fact *= n
    n -= 1
print(fact)
```

OUTPUT

```
120
```

Factorials from 1 to 6

Here a loop inside a loop works out many factorials. Notice that `fact = 1` is **reset for every number**.

```
for n in range(1, 7):
    fact = 1
    for i in range(1, n + 1):
        fact *= i
    print(n, "! =", fact)
```

OUTPUT

```
1 ! = 1
2 ! = 2
3 ! = 6
4 ! = 24
5 ! = 120
6 ! = 720
```

Ask

What would go wrong if `fact = 1` was written before the outer loop instead?

Prime numbers

A **prime number** is greater than 1 and divisible only by **1 and itself**: 2, 3, 5, 7, 11, 13 ...

Idea: try to divide `n` by every number from 2 up to `n - 1`. If **any** divides it, it is **not prime**.

Is n prime?

```
n = 29
is_prime = n > 1
for i in range(2, n):
    if n % i == 0:
        is_prime = False
        break
if is_prime:
    print(n, "is prime")
else:
    print(n, "is not prime")
```

OUTPUT

```
29 is prime
```

- `is_prime` is a **flag**: it starts as True and flips to False if a divisor is found.
- `break` stops at the **first divisor**, because one is enough.
- `n > 1` makes sure 0 and 1 are not called prime.

A faster check

A divisor can only be as big as the **square root** of `n`, so you can stop there.

```
n = 97
is_prime = n > 1
for i in range(2, int(n ** 0.5) + 1):
    if n % i == 0:
        is_prime = False
        break
print(n, "is prime" if is_prime else "is not prime")
```

OUTPUT

```
97 is prime
```

All primes up to 30

```
for n in range(2, 31):
    is_prime = True
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            is_prime = False
            break
    if is_prime:
        print(n, end=" ")
```

OUTPUT

```
2 3 5 7 11 13 17 19 23 29
```

Fibonacci series

Each number is the **sum of the two before it**: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34 ...

First 10 terms

```
a, b = 0, 1
for _ in range(10):
    print(a, end=" ")
    a, b = b, a + b
```

OUTPUT

```
0 1 1 2 3 5 8 13 21 34
```

- `a` and `b` hold the **current and next** terms.
- `a, b = b, a + b` updates **both at once**, so no third variable is needed.
- `_` is a loop variable we do not use. It is a common name for that.

Fibonacci while a value stays small

```
a, b = 0, 1
while a < 100:
    print(a, end=" ")
    a, b = b, a + b
```

OUTPUT

```
0 1 1 2 3 5 8 13 21 34 55 89
```

Ask

Which loop is better when you want exactly 10 terms? Which when you want all terms below 100?

More number programs

Reverse a number and check a palindrome

```
n = 1221
original = n
rev = 0
while n > 0:
    rev = rev * 10 + n % 10
    n //= 10
print(rev)
if rev == original:
    print("Palindrome number")
else:
    print("Not a palindrome")
```

OUTPUT

```
1221
Palindrome number
```

Each round takes the **last digit** (`n % 10`), adds it to the **end of** `rev` , and drops it from `n` (`n //= 10`).

GCD (greatest common divisor)

```
a, b = 48, 18
while b != 0:
    a, b = b, a % b
print("GCD =", a)
```

OUTPUT

```
GCD = 6
```

This is **Euclid method**: replace the pair with (smaller, remainder) until the remainder is 0.

Tips and common mistakes

MISTAKE	WHAT HAPPENS	FIX
Forgetting `print()` after the inner loop	the whole pattern is on one line	add `print()` at the outer loop level
`range(1, n)` when you need n	the last row or number is missing	use `range(1, n + 1)`
Starting `fact = 0`	the answer is always 0	start the product at 1
Setting `fact = 1` outside the outer loop	the numbers keep growing across rounds	reset it inside the outer loop
Wrong indentation of `print()`	prints too often or only once	check which loop it belongs to

Pattern without the new line

Try it: add the missing print() so each row gets its own line

```
for i in range(1, 5):
    for j in range(1, i + 1):
        print("*", end="")
```

OUTPUT

```
**********
```

Practice questions

Pattern questions

1. Print a square of stars with 4 rows and 4 columns.
2. Print a right triangle of numbers: `1`, `1 2`, `1 2 3`, ...
3. Print an inverted triangle of numbers: `1 2 3 4 5`, `1 2 3 4`, ...
4. Print a pyramid of stars with 5 rows.
5. Print a hollow square of stars (stars only on the border).

Number program questions

6. Take a number and print its factorial.
7. Take a number and check whether it is prime.
8. Print all prime numbers between 1 and 100.
9. Print the first `n` Fibonacci numbers.
10. Take a number and check whether it is a palindrome.

Find the output

Work out the output first, then press Run to check

```
for i in range(1, 4):
    for j in range(i):
        print(i, end=" ")
    print()
```

Try a pattern

Your turn: print a square of 4 x 4 stars, then press Run

```
# use two loops and print()
```

Homework and wrap-up

- Print a **diamond** of stars (a pyramid followed by an inverted pyramid).
- Print **Armstrong numbers** between 1 and 1000 (a number is an Armstrong number if the sum of the cubes of its digits equals the number, for example 153).
- Find all **perfect numbers** up to 1000 (a number equal to the sum of its divisors smaller than itself, for example 6 = 1 + 2 + 3).
- Take two numbers and print all **prime numbers between them**.
- Print the **multiplication tables from 1 to 5** side by side using nested loops.

Next class

Lists.